Reaction Diffusion Systems and Turing Patterns

Ragav Srinivasan

November 11, 2026

Abstract

Procedural generation algorithms such as Perlin Noise are widespread in the Games and Entertainment industry. These algorithms can produce pseudo-realistic visuals of landscapes and natural vegetaion. However, they cannot produce organized patterns. Reaction Diffusion systems have been studied widely ever since Turing's paper on chemical morphogenesis which showed pattern formation resembling those found in biology. In this paper, we give a description turing patterns and study the patterns arising from the Gray Scott Model. I have done a Numerical simulation of the Gray Scott using a method provided by J.E Pearson to reproduce these patterns. Although not complete, the report intends to show how patterns can be used in computer graphics.

1 Introduction

This report is an exploration of Pattern formation from Reaction Diffusion Systems.

Section 2 gives the necessary context and background on this report, this includes a brief overview of procedural generation by Perlin Noise and it's limitations, an introduction to reaction diffusion systems, a discussion on Turing Patterns, and finally we will focus on pattern formation arising from the Gray Scott Model.

?? is my original contribution towards this topic featuring my stability analysis of the Gray Scott Model, A Numerical Simulation implementing a method proposed by (Pearson 1993) and finally a construction of a parameter map inspired by (Munafo 2022).

Section 4 summarizes the research in this report and proposes the planned extensions of this topic that were unable to be featured this report.

2 Background

2.1 Procedural Generation

Procedural generation is a broad set of techniques in computer graphics that is used to create visuals, such as pseudo-realistic textures, terrains and surfaces on the fly. It eliminates the need for artists to manually prototype and create their own worlds. Instead, a deterministic function called *noise* is used to produce the desired visuals, on the fly as the player explores. This makes it possible to build large, *lifelike* landscapes with mountains, rivers and vegetation with little impact on memory (Lagae et al. 2010).

Procedural generation is based on many algorithms. In this section we study one of the most widely used algorithms: *Perlin noise*.

Ken Perlin devised his noise function after working on the visual effects for Disney's 1982 film *Tron* at Mathematical Applications Group, Inc. (MAGI), and published it in 1985 (Perlin 1985). His technique was so pivotal to the industry that it earned him an Academy Award for Technical

Achievement (Perlin 2026), and it is still widely used in modern-day films and video games (Lagae et al. 2010).

Although, Perlin Noise has widespread use in the industry, There are some limitations. Textures created with Perlin noise are random and have no structure (Lagae et al. 2010). Although this approach is ideal for terrain generation, it cannot be used to generate patterns found in biology such as coral reefs, zebra stripes, giraffe spots, etc. For this we use a different technique found in Reaction Diffusion Systems.

2.2 Reaction Diffusion Systems

This brings us to the heart of this report: *reaction-diffusion systems*. These are nonlinear dynamical systems, originally from chemistry, and models changes in concentrations of several chemical substances as they react and diffuse over space and time (Turing 1952).

Diffusion refers to the behaviour of chemical substances *spreading* from regions of high concentration to low concentration and as a result the substance is uniformly distributed across a medium (Encyclopædia Britannica 2026). It turns out that mathematically modelling this behaviour leads to an interesting variety of pattern formation called Turing patterns, which will be the focus of Section 2.3.

The general form of a reaction-diffusion system is

$$\frac{\partial \mathbf{q}}{\partial t} = D\nabla^2 \mathbf{q} + R(\mathbf{q}), \tag{2.1}$$

Above is a dimensionless equation wherein all the physical quantities are stripped and all the terms scaled (Strogatz 2015).

The equation is above is dimensionless which means the physical quantities are stripped. This reduces the equation to include only a few parameters F and k making analysis much easier. (Strogatz 2015).

2.3 Turing patterns

In 1952, towards the end of Alan Turing's life, Turing publishes one of his most seminal contributions in the field of mathematical biology. In his 1952 paper titled "The Chemical Basis of Morphogenesis", Turing discovers a mechanism which describes the formation of some patterns found in nature (Turing 1952). For instance, the patterns found in animal skin such as the stripes of a zebra, spots of a jaguar, the structure of our brains and the formation of coral reefs are considered to be Turing patterns (University of Sheffield 2022).

In the paper, Turing studies the development of an embryo and describes a system of signalling molecules called *morphogens*. The reaction and diffusion shown by these morphogens can destabilise the system leading to an interesting case of pattern formation (Turing 1952).

In the following section, we will be studying Turing patterns formed by a simple model called the Gray-Scott model .

2.4 The Gray-Scott model

The setup for the Gray-Scott model supposes that we have a *well-mixed tank* of chemical U. Then the concentration of U. We are interested in knowing the concentration of the chemical in space and time. For this, we define a vector function $u(x,t)$ that is currently unknown. We assume that chemical U has the same concentration in the spatial domain except for a small patch in the center of the tank where chemical V is introduced into the system.

In chemistry, the relationship between U and V is defined as *autocatalysis* where the resulting product is a reactant, as is the case with V in the reaction shown below. The relationship is akin to that of *prey and predator* models.

$$\underbrace{U + 2V \rightarrow 3V}_{V \text{ reacts with } U \text{ to make more of itself}} \qquad \underbrace{V \rightarrow P}_{\text{decay}}$$

The second rule suggests decay for V by turning into an inert product P which means that it becomes a non-reactive substance.

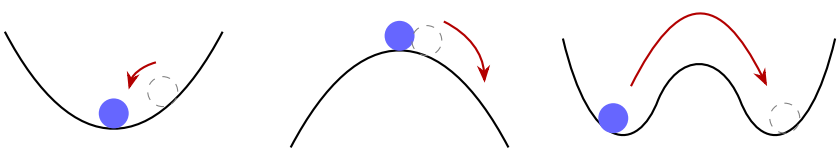
$$\frac{\partial U}{\partial t} = D_u \nabla^2 U - UV^2 + F(1 - U)$$
$$\frac{\partial V}{\partial t} = D_v \nabla^2 V + UV^2 - (F + k)V$$

- D_u and D_v are the diffusivities of chemicals U and V.
- U and V are the unknown functions representing the concentrations of the respective chemicals in space and time.
- F is the feed rate of chemical U.
- k is the decay rate for chemical V.
- ∇^2 is the laplacian operator calculates the average concentration of neighbouring molecules.

3 Original Contribution

3.1 Linear Stability Analysis of the Gray-Scott Model

Steady states of a dynamical system are states after which point the system shows no change. We consider a small disturbance around a steady state and say the state is *stable* if the disturbance dampens and the system loops back to its steady state. While an *unstable* steady state is when the disturbance grows and thus the system never returns back to its steady state (Teschl 2012). To give an analogy, Figure 1 compares a ball on a hilltop versus in a valley. Panel (a) is a stable state as the ball returns to its steady state after a small nudge. Panel (b) is an example of an unstable state, the ball never returns to a steady state after a small push and finally panel (c) shows a bistable state where the ball returns to a steady state but at a different point.



(a) Stable (b) Unstable (c) Bistable

Figure 1: An analogy for Stability Analysis of Steady States

Nonlinear systems such as the Gray-Scott model are difficult to solve analytically. Therefore, I will be doing a stability analysis to find out the behaviour of the system near its steady states. To do this, we will first linearise the system by its Jacobian matrix and classify by its trace and determinant.

In this analysis I will show how diffusion can *destabilise* a steady state and result in pattern formation. The analysis will be done twice, once by dropping the coefficients of diffusion D_u and D_v and then we will add them back to the system and check whether it destabilises the system.

$$\begin{aligned}\frac{\partial U}{\partial t} &= D_u \nabla^2 U + \underbrace{-UV^2 + F(1-U)}_{f(U,V)},\\ \frac{\partial V}{\partial t} &= D_v \nabla^2 V + \underbrace{UV^2 - (F+k)V}_{g(U,V)}.\end{aligned} \tag{3.1}$$

Stability Analysis without Diffusion

Setting $D_u = D_v = 0$ in Equation (3.1), every point in space will behave in the same way. We define $\dot{U}$ and $\dot{V}$:

$$\begin{aligned}\dot{U} &= f(U,V) = -UV^2 + F(1-U),\\ \dot{V} &= g(U,V) = UV^2 - (F+k)V.\end{aligned} \tag{3.2}$$

A steady state is when $f = g = 0$. Setting $g = 0$ and factorizing by V gives us $g = V(UV + (F+k)) = 0$. Here, we have two cases for V. If $V = 0$, then $U = 1$ which is our *trivial steady state*.
Let's first classify stability of the trivial steady state by the Jacobian of the system given by:

$$J = \begin{pmatrix} \frac{\partial f}{\partial U} & \frac{\partial f}{\partial V} \\ \frac{\partial g}{\partial U} & \frac{\partial g}{\partial V} \end{pmatrix} = \begin{pmatrix} -V^2 - F & -2UV \\ V^2 & 2UV - (F+k) \end{pmatrix}. \tag{3.3}$$

To classify the steady state, we first substituted the point into the above Jacobian. Then calculate, $p = trJ$ and $q = detJ$. The state is considered to be stable if $p < 0$ and $q > 0$.

$$J(1,0) = \begin{pmatrix} -F & 0 \\ 0 & -(F+k) \end{pmatrix}, \qquad p = -(2F+k), \qquad q = F(F+k), \qquad p^2 - 4q = k^2. \tag{3.4}$$

Since $F, k > 0$, we have $p < 0$, $q > 0$ and $p^2 - 4q \geq 0$, so the trivial state is a *stable node for every* F and k. This suggests that when a small concentration of chemical V is added to the tank mixed by U, the V dies out and the system is back to its original state.

Next, we consider the second case where $UV + (F+k) = 0 \rightarrow UV = (F+k)$. Substituting $UV = (F+k)$ in $f(U,V)$ gives the set of non-trivial steady states:

$$\begin{aligned} V &= \frac{F(1-U)}{F+k}, \qquad U^2 - U + c = 0 \quad \text{with } c = \frac{(F+k)^2}{F},\\ U_\pm &= \frac{1}{2}\left(1 \pm \sqrt{1 - \frac{4(F+k)^2}{F}}\right), \qquad V_\pm = \frac{F+k}{U_\pm}. \end{aligned} \tag{3.5}$$

The steady states only exist under the following condition:

$$F \geq 4(F+k)^2 \Rightarrow k \leq \tfrac{1}{2}\sqrt{F} - F. \tag{3.6}$$

(Not complete) Doing further analysis from here will show that the non-trivial steady is stable.

Adding back diffusion

Not complete.

3.2 Simulation of the Gray–Scott Model

In *Complex Patterns in a Simple System*, Pearson (1993) numerically simulated the Gray-Scott model and found a wide variety of patterns across the (F, k) parameter space, including Turing-type patterns such as hexagonal arrays of spots. He integrated the equations using the forward Euler method on a 256×256 grid and classified the resulting patterns into 14 classes.

Because Pearson (1993) simulated the system with an initial condition consisting of a small square in the center of the grid with $(U, V) = (\frac{1}{2}, \frac{1}{4})$, his simulation missed several types of pattern. This observation was made by Robert Munafo (Munafo 2022), who extended Pearson's simulations with other starting conditions and plotted a parameter map of F, k values and showed that there are actually 19 classes of patterns in the Gray-Scott model.

Inspired by the work of Pearson (1993) and Munafo (2022), I will be implementing my own simulation using Pearson's Method to produce some examples of these patterns. I will also build my own version of Munafo's parameter map to show which patterns the Gray-Scott model produces with which parameters.

I implemented the simulation in a Python Notebook using NumPy. The Laplacian is computed for the whole grid at once with a five point stencil. The values for the diffusion rates are $D_u = 0.2$ and $D_v = 0.1$ and a time step of 0.1. Following Pearson, each run starts from the square of $(U, V) = (\frac{1}{2}, \frac{1}{4})$ at the centre of the grid with 1%

I implemented the simulation in a Python Notebook using Numpy and plotted the results with Matplotlib (Code is given in the appendix). The parameters for the simulation are $D_u = 0.2$ and $D_v = 0.1$ with a time step of $dt = 1$. The initial conditions in the simulation are the same as Pearson's where the concentrations of U and V in the center of the grid are 0.5 and 0.25 respectively. I then chose the parameter $F = 0.022, k = 0.051$ value from (Munafo 2022).

Figure 2 shows the result of a stepwise simulation using parameters $F = 0.022, k = 0.051$.

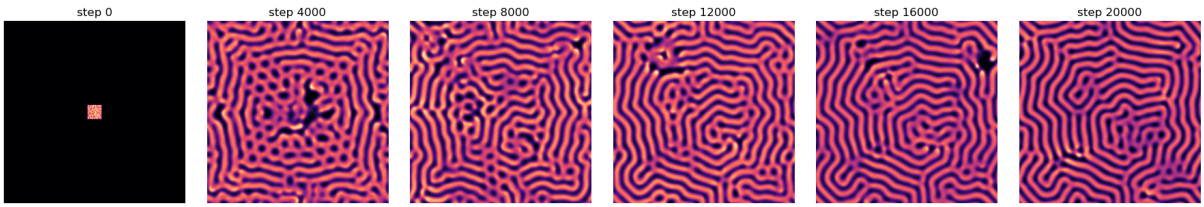


Figure 2: Simulation of the Gray Scott model on $F = 0.022, k = 0.051$

The results show a coral like reef like pattern.

3.3 Parameter Map

Not complete

4 Conclusion

In conclusion, This report looked at a widely used technique in Computer Graphics called *Perlin Noise* which is used to produce pseudo-realistic graphics of natural landscapes, vegetation and surfaces such as marble, wood, etc. Although a very useful algorithm, it's visuals are random and lack structure and is not the ideal algorithm to generate textures in biology. For

this, we introduce Reaction Diffusion systems which is shown produce certain patterns that resemble biology, followed by a discussion of Alan Turing's 1952 paper on the "Chemical Basis of Morphogenesis" where he proposed a mechanism for pattern formation in biology by adding diffusion to a system modelling the development of an embryo. Finally, we studied the Gray-Scott model

Due to time constraints, the report does not cover the intended goal of applying Reaction Diffusion systems to Computer Graphics. However, it is possible to use the results of from our numerical simulation in creating texture maps that can be used in 3D Design and Game development tools such as Blender, Godot, Unity and Unreal Engine. This will enable artists to produce graphics of natural phenomenon whose patterns can be explained by Reaction Diffusion.

This report also attempts to do a stability Analysis on the Gray Scott Model with Diffusion. An analysis without the diffusion terms was done in this report which showed that the steady states of the model were stable. However, further analysis after adding back diffusion terms is still pending. I hypothesize that adding back diffusion to the system should destabilize the system as shown by (Turing 1952), I believe this analysis could be helpful in identifying which parameters lead to stable and unstable states.

Additionlly, a parameter map was also planned but the section stands incomplete due to falling short of time.

References

Encyclopædia Britannica (2026). *Diffusion*. URL: https://www.britannica.com/science/diffusion (visited on 11/11/2026).

Lagae, Ares et al. (2010). "A Survey of Procedural Noise Functions". In: *Computer Graphics Forum* 29.8, pp. 2579–2600. DOI: 10.1111/j.1467-8659.2010.01827.x.

Munafo, Robert P. (2022). *Pearson's Classification (Extended) of Gray-Scott System Parameter Values*. MROB. URL: https://www.mrob.com/pub/comp/xmorphia/pearson-classes.html (visited on 11/11/2026).

Pearson, John E. (1993). "Complex Patterns in a Simple System". In: *Science* 261.5118, pp. 189–192. DOI: 10.1126/science.261.5118.189.

Perlin, Ken (1985). "An Image Synthesizer". In: *Computer Graphics (Proceedings of SIGGRAPH '85)* 19.3, pp. 287–296. DOI: 10.1145/325165.325247.

— (2026). *Noise and Turbulence*. URL: https://mrl.cs.nyu.edu/~perlin/doc/oscar.html (visited on 11/11/2026).

Strogatz, Steven H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. 2nd ed. Boulder, CO: Westview Press.

Teschl, Gerald (2012). *Ordinary Differential Equations and Dynamical Systems*. Vol. 140. Graduate Studies in Mathematics. Providence, RI: American Mathematical Society. URL: https://www.mat.univie.ac.at/~gerald/ftp/book-ode/.

Turing, Alan M. (1952). "The Chemical Basis of Morphogenesis". In: *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences* 237.641, pp. 37–72. DOI: 10.1098/rstb.1952.0012.

University of Sheffield (2022). *Turing Patterns*. Turing Pattern Project. URL: https://turing-pattern-project.sites.sheffield.ac.uk/turing-patterns (visited on 11/11/2026).

5 Project Plan

Project plan as of 23rd October

Project title

Reaction Diffusion Systems and Turing Patterns

Overview

This research paper will focus on the applications of Reaction Diffusion Systems. Reaction Diffusion systems emerge from Chemistry and Biology, however have found use in a lot of areas. The project will provide a background to Reaction Diffusion Systems, explore real world applications and document my own application for using it as a procedural generation algorithm (Python Program).

Required reading and sources

To make it easier to start my research, I have structured my reading by effort: *low effort*, *medium effort* and *high effort*.

1. *Low effort*: Short reading times of 15 to 30 minutes, aimed to give a basic understanding of the topic I'm researching. Videos from YouTube are also included in this category:
    - Wikipedia: Reaction–diffusion system and Turing pattern: overview of the model and its biological motivation.
    - Coding Train: Coding Challenge #13, Reaction Diffusion Algorithm (YouTube): live implementation of the Gray–Scott model in p5.js.
    - Karl Sims: Reaction-Diffusion Tutorial: intuitive explanation of feed/kill rates, the discrete Laplacian, and the (k, f) parameter map.
2. *Medium effort*: Semi-long reading times of 30 to 45 minutes, aimed to give a more technical understanding of the topic with definitions and theorems related to the topic. Reading material includes longer videos, tutorials for math concepts from textbooks / internet, etc.
    - Robert Munafo: Reaction-Diffusion by the Gray-Scott Model, Pearson's Parametrization: term-by-term breakdown of the PDEs and a classified map of pattern types across parameter space.
    - Red Blob Games: Turing Patterns notes: worked exploration of Gray–Scott, including parameter sensitivity and convention differences between sources.
    - Numerical MOOC: Reaction-Diffusion notebook: forward-time/central-space discretisation of Gray–Scott in Python.
    - Strogatz, *Nonlinear Dynamics and Chaos* (2nd ed.), Ch. 5–6 (linear systems, phase plane) and Ch. 8 (bifurcations in 2D): the linearisation and trace–determinant tools needed for Turing stability analysis.
    - Kondo & Miura (2010), Reaction-diffusion model as a framework for understanding biological pattern formation, *Science*: accessible review connecting the theory to real animal patterns.
3. *High effort*: Long reading times of up to 60 minutes. The reading material is much more complex and targeted, requires a lot of effort to understand. Material includes research papers, advanced math topics and full lectures:

- Turing (1952), The Chemical Basis of Morphogenesis, *Phil. Trans. R. Soc. B*: the original diffusion-driven instability argument.
- Pearson (1993), Complex Patterns in a Simple System, *Science* 261: the Gray–Scott parameter classification used by most graphics implementations.
- Turk (1991), Generating Textures on Arbitrary Surfaces Using Reaction-Diffusion, SIGGRAPH '91: reaction-diffusion applied directly to 3D surfaces.
- Witkin & Kass (1991), Reaction-Diffusion Textures, SIGGRAPH '91: anisotropic and spatially varying reaction-diffusion for texture synthesis.
- Murray, *Mathematical Biology II: Spatial Models and Biomedical Applications* (3rd ed.), Ch. 2: full derivation of the Turing instability conditions and the critical wavenumber, which is the core of the linear stability analysis.

What I'll do

The research project will start with an introduction of the chosen research topic: *Reaction Diffusion Systems* and *Turing Patterns*. Then I plan to investigate the solutions of this family of equations and determine the behaviour. After this, I'm planning to highlight several applications of this topic that I've found from my research. In the end, I would like to wrap up this report with my original contribution of developing a program in Python that is capable of simulating and rendering patterns from Reaction Diffusion Systems.

Original contribution

My original contribution to this report will include using the knowledge I've gained from this research topic and apply it to develop a Python program capable of producing Turing patterns.

Timeline

Timeline as of 30th October:

- ☒ Revised project plan, timeline and gather reading material
- ☒ Read material, take notes and make timeline more specific
- ☒ Read more material, take notes and write everything I've learned from my reading
- ☒ Define the right scope for the project and start thinking about ideas for original contribution
- ☒ 30th October: Research and take notes
- ☐ Focus on original contribution
- ☐ Finish original contribution by 1st November
- ☐ Send project for feedback
- ☐ Act on feedback and revise original contribution
- ☐ Complete introduction and conclusion sections by 5th November
- ☐ Send project for feedback before next week's class
- ☐ Proofread the project, look for issues
- ☐ Fix those issues
- ☐ 8th November: Project submission

Version control

The research project is tracked using GitHub. The repository of the project can be accessed at: https://github.com/Rageon787/math3030-project.git

Plan revisions

Revised plan with new timeline as I've fallen behind a lot. Future revisions will be about the scope of the project and original contribution.

6 Reflection

When I first started working on this project, I had set myself some high expectations on this project. In the end, the plan I gave myself turned out to be unrealistic. I made revisions to the project each time I fell behind on progress. The final update to the project plan was on 23rd October as at the time I felt like I had a firm idea for my report. The checklists i made for myself didn't turn out to be realistic given my speed on this project and my workload from other units.

I chose this topic because of my interest in Computer Graphics enjoy its use for mathematics and found Reaction Diffusion Systems to be link between nonlinear dynamics and computer graphics. In hindsight, I'm not sure if it turned out to be a good idea do something that interested as it created a lot of bias in the research.

I planned the report to be a study on a specific Reaction Diffusion model called the Gray Scott Model. For the original contribution, I wanted to do a stability analysis and my own simulation of the Grey Scott Model using a method described by Pearson in his paper *Complex Patterns in a Simple System* (Pearson 1993). However, when i started doing the stability analysis for the Gray Scott Model with Diffusion I personally found it to be very difficult. So in my report, I've only been able to do a stability analysis of a single aspect of the model.

I planned my report to be a study on Turing Patterns formed by Reaction Diffusion Systems. There is a fair amount of literature on the topic of Turing Patterns including Turing's discovery of these patterns in 1952, Pearson's numerical simulation (Pearson 1993) on the Gray Scott Model.

I'm not very proud with the current state of my report as it stands incomplete and there are a lot of unanswered research questions. Although, I have stopped working on this report now. I find the topic fascinating so I've made the commitment to keep learning more about this topic in preparation for my viva exam and beyond. Specifically, I want to fill the gap in my stability analysis in which I couldn't complete further analysis of the non-trivial steady states and the part of adding back the diffusion coefficients.

I deeply regret not asking for a lot of feedback on my project. I didn't feel confident send my report as I felt it wasn't coherent enough to read. I also spent a majority writing text for my report in a notebook as I felt it was much faster than typing.

On a positive note, Doing this research project has given me a better grasp of Nonlinear Dynamics and the numerical methods for solving a system of Partial Differential Equations that I learned last semester.

7 Appendix

This appendix contains the Python code used for the simulation in **??**.

7.1 Initial conditions

```
## Parameters
N = 256 ## Grid size
Du, Dv, dt = 0.2, 0.1, 1.0
F, k = 0.022, 0.051 ## Feed rate and kill rate

## Seed for random number generator
rng = np.random.default_rng(0)


```

```
## Concentration of U and V in each cell
U = np.ones((N, N))  ## Initially U is one everywhere
V = np.zeros((N, N))  ## and V is zero everywhere

## Except, in the center of the grid, where (U, V) = (0.5, 0.25)
center_u, center_v = 0.5, 0.25
## let c be the center point of the grid
c = N // 2
## Then we slice the center grid
center_grid = (slice(c - 10, c + 10), slice(c - 10, c + 10))   # 20x20 block
U *= rng.uniform(0.95, 1.05, size=(N, N))
V *= rng.uniform(0.95, 1.05, size=(N, N))


## This center grid will have (U,V) = (0.5, 0.25)
V[center_grid] = rng.uniform(center_v, center_u, size = (20, 20))
U[center_grid] = 1 - 2 * V[center_grid]
```

7.2 Laplacian and forward Euler step

```
## Laplacian function that calculates the average concentration of a cell's neigbours

import numpy as np
from scipy.ndimage import convolve #

# 5 point stencil for the laplacian
stencil5 = np.array([[0,  1, 0],
                     [1, -4, 1],
                     [0,  1, 0]])


def laplacian(Z):
    return convolve(Z, stencil5, mode='wrap')  # laplacian
## Forward Euler Integration
def step(U, V):

    lapU = laplacian(U)
    lapV = laplacian(V)
    uvv = U * V * V                       # the reaction term UV^2, computed once

    U_new = U + dt * (Du * lapU - uvv + F * (1 - U))
    V_new = V + dt * (Dv * lapV + uvv - (F + k) * V)
    return U_new, V_new


# Simulate 20000 times, with a snapshot of the system for every 4000 steps
n_steps = 20000
snap_every = 4000 # Snapshot interval
snapshots = []

for i in range(n_steps + 1):
    if i % snap_every == 0: ## Is this a multiple of 4000?
        snapshots.append((i, V.copy())) ## Append this figure to the list of snapshots
    U, V = step(U, V) ## Move to the next step in the integration



## Code for making figures
fig, axes = plt.subplots(1, len(snapshots), figsize=(3 * len(snapshots), 3))
for ax, (i, snap) in zip(axes, snapshots):
    ax.imshow(snap, cmap="magma")
    ax.set_title(f"step {i}")
```

```
    ax.axis("off")
plt.tight_layout()
plt.show()

print("V range:", V.min(), V.max())
```

7.3 Parameter map

```
## ---- Parameter map (Pearson's method, shortened) ----
import time
import numpy as np
import matplotlib.pyplot as plt

GRID_SIZE = 96          # grid size for the map (smaller than Pearson's 256)
step_size = 15000   # Step size for euler integration

## Simulates the Gray Scott model with chosen paramters using Pearson's Method
def simulate(F, k, n=GRID_SIZE, n_steps=step_size, seed=0):
    rng = np.random.default_rng(seed)
    U = np.ones((n, n)) ## Iniitally U has full concentration everywhere
    V = np.zeros((n, n)) ## No V in the system
    c, h = n // 2, 5
    Du, Dv, dt = 0.2, 0.1, 1.0

    center_square = (slice(c - h, c + h), slice(c - h, c + h)) # 10 x 10 central square in the
    grid
    U[center_square], V[center_square] = 0.5, 0.25 #
    U *= rng.uniform(0.99, 1.01, size=(n, n))          # vary concentraiton of U and V in the
    grid by + or - 1 percent
    V *= rng.uniform(0.99, 1.01, size=(n, n))
    for _ in range(n_steps):
        uvv = U * V * V
        U = U + dt * (Du * laplacian(U) - uvv + F * (1 - U))
        V = V + dt * (Dv * laplacian(V) + uvv - (F + k) * V)
    return V

## Creates a grid of (F, k) values
k_values = np.linspace(0.04, 0.08, 8)
F_values = np.linspace(0.01, 0.07, 6)

## Run every combination, keeping the final V grid of each
results = {} ## a dictionary containing the final V values for each (F, k) pair
t0 = time.time()
for F_ in F_values:
    for k_ in k_values:
        results[(F_, k_)] = simulate(F_, k_)
    print(f"F = {F_:.3f} done  ({time.time() - t0:.0f} s)") ## Debug code


## Figure related code that I didn't write
np.savez("figures/parameter_map_data.npz",
         F=F_values, k=k_values,
         V=np.array([[results[(F_, k_)] for k_ in k_values] for F_ in F_values]))

## Draw all tiles on one set of axes, using real (k, F) coordinates
dk, dF = k_values[1] - k_values[0], F_values[1] - F_values[0]
fig, ax = plt.subplots(figsize=(9, 7))
for (F_, k_), V_ in results.items():
    ax.imshow(V_, cmap="magma", vmin=0, vmax=0.5, origin="lower",
              extent=[k_ - dk / 2, k_ + dk / 2, F_ - dF / 2, F_ + dF / 2],
              aspect="auto")

```

```
## Stability analysis: non-trivial steady states exist only for k <= sqrt(F)/2 - F
F_line = np.linspace(F_values[0] - dF / 2, F_values[-1] + dF / 2, 300)
ax.plot(0.5 * np.sqrt(F_line) - F_line, F_line, "w--", lw=1.5,
        label=r"$k = \frac{1}{2}\sqrt{F} - F$")

ax.set_xlim(k_values[0] - dk / 2, k_values[-1] + dk / 2)
ax.set_ylim(F_values[0] - dF / 2, F_values[-1] + dF / 2)
ax.set_xticks(k_values); ax.set_xticklabels([f"{x:.3f}" for x in k_values])
ax.set_yticks(F_values); ax.set_yticklabels([f"{x:.3f}" for x in F_values])
ax.set_xlabel("kill rate $k$"); ax.set_ylabel("feed rate $F$")
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig("figures/parameter_map.png", dpi=200)
plt.show()
```